CSE 6220/CX 4220
Introduction to HPC

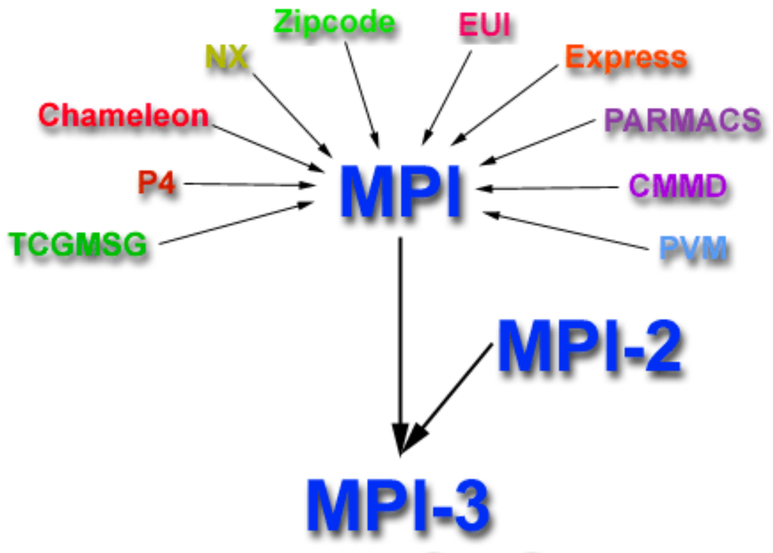
Lecture 4: Introduction to MPI

Helen Xu
hxu615@gatech.edu

Slides from UC Berkeley 267, Srinivas Aluru

Message Passing Interface (MPI)

- In the 1980s-early 90s, there used to be many software tools for writing distributed-memory programs.
- 1994 - MPI 1.0 is released - works on distributed and shared memory
- Standards are needed to write portable code.



Message Passing Libraries

All **communication, synchronization require subroutine calls**

- **No shared variables**
- Program run on a single processor just like any uniprocessor program, except for calls to message passing library

Subroutines for

- **Communication**
  - Pairwise or point-to-point: Send and Receive
  - Collectives all processor get together to
    Move data: Broadcast, Scatter/gather
    Compute and move: sum, product, max, prefix sum, … of data on many processors
- **Synchronization** (barrier)
- **Enquiries** - How many processes? Which one am I? Any messages waiting?

Compile/Run MPI

In your code:

```
#include <mpi.h>

rc = MPI_XXXX(parameter, …)
```

To compile:

```
$ mpicc mpi_hello_world.c -o mpi_hello_world
```

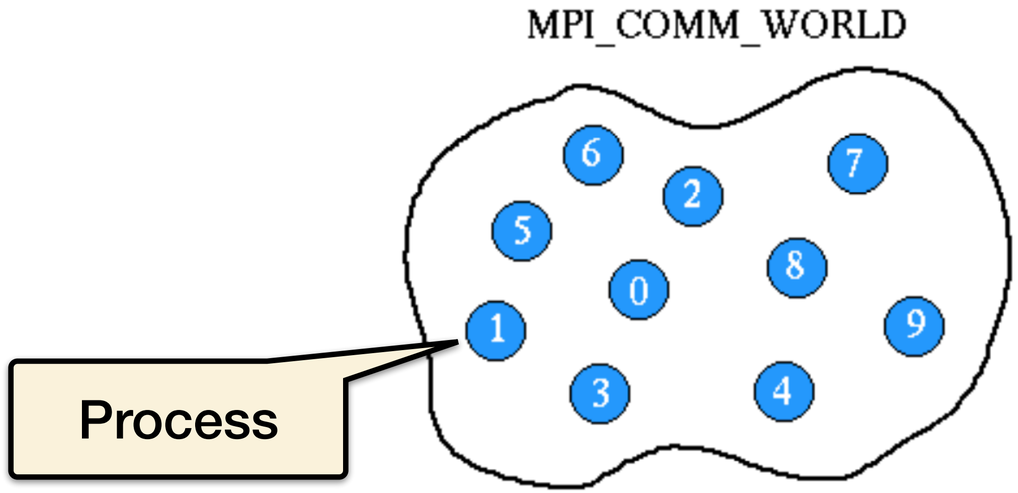
To run:

```
$ srun mpi_hello_world
```

(The default implementation of MPI on PACE ICE is mvapich2.
Different implementations (e.g., OpenMPI) may have slightly different usage)

MPI Communicator

- An **MPI communicator**: a "communication universe" for a group of processes
- `MPI_COMM_WORLD` – name of the default MPI communicator, i.e., the collection of all processes
- Each process in a communicator is identified by its **rank**
- Almost every MPI command needs to provide **a communicator as input argument**



https://www.uio.no/studier/emner/matnat/ifi/INF3380/v11/undervisningsmateriale/inf3380-week06.pdf

MPI Process Rank

- Each process has a unique **rank**, i.e. an integer identifier, within a communicator
- The rank value is between `0` and `#procs-1`
- The rank value is used to distinguish one process from another
- Commands `MPI_Comm_size` & `MPI_Comm_rank` are very useful

Example:

```
int size, my_rank;

MPI_Comm_size (MPI_COMM_WORLD, &size);
MPI_Comm_rank (MPI_COMM_WORLD, &my_rank);

if (my_rank==0) { ... }
```

The 6 Most Important MPI Commands

- `MPI_Init` - initiate an MPI computation
- `MPI_Finalize` - terminate the MPI computation and clean up
- `MPI_Comm_size` - how many processes participate in a given MPI communicator?
- `MPI_Comm_rank` - which one am I? (A number between 0 and size-1.)
- `MPI_Send` - send a message to a particular process within an MPI communicator
- `MPI_Recv` - receive a message from a particular process within an MPI communicator

MPI Hello World Example

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size); // Get the number of processes

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank); // Get the rank of the process

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len); // Get the name of the processor

  printf("Hello world from processor %s, rank %d out of %d processors\n",
         processor_name, world_rank, world_size); // Print hello world message

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

https://raw.githubusercontent.com/mpitutorial/mpitutorial/gh-pages/tutorials/mpi-hello-world/code/mpi_hello_world.c

MPI Hello World Example

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size); // Get the number of processes

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank); // Get the rank of the process

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len); // Get the name of the processor

  printf("Hello world from processor %s, rank %d out of %d processors\n",
         processor_name, world_rank, world_size); // Print hello world message

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

Initialize enironment

https://raw.githubusercontent.com/mpitutorial/mpitutorial/gh-pages/tutorials/mpi-hello-world/code/mpi_hello_world.c

MPI Hello World Example

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size);

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank); // Get the rank of the process

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len); // Get the name of the processor

  printf("Hello world from processor %s, rank %d out of %d processors\n",
         processor_name, world_rank, world_size); // Print hello world message

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

Initialize enironment

How many processors are there?

https://raw.githubusercontent.com/mpitutorial/mpitutorial/gh-pages/tutorials/mpi-hello-world/code/mpi_hello_world.c

MPI Hello World Example

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size);

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len); // Get the name of the processor

  printf("Hello world from processor %s, rank %d out of %d processors\n",
         processor_name, world_rank, world_size); // Print hello world message

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

Initialize enironment

How many processors are there?

What is my rank?

https://raw.githubusercontent.com/mpitutorial/mpitutorial/gh-pages/tutorials/mpi-hello-world/code/mpi_hello_world.c

MPI Hello World Example

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size);

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len);

  printf("Hello world from processor %s, rank %d out of %d processors\n",
         processor_name, world_rank, world_size); // Print hello world message

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

Initialize enironment

How many processors are there?

What is my rank?

What is my name?

https://raw.githubusercontent.com/mpitutorial/mpitutorial/gh-pages/tutorials/mpi-hello-world/code/mpi_hello_world.c

MPI Hello World Example

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size);

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len);

  printf("Hello world from processor %s, rank %d out of %d processors\n",
         processor_name, world_rank, world_size);

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

Initialize enironment

How many processors are there?

What is my rank?

What is my name?

Print name, rank, total processor count

https://raw.githubusercontent.com/mpitutorial/mpitutorial/gh-pages/tutorials/mpi-hello-world/code/mpi_hello_world.c

MPI Hello World Example

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size);

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len);

  printf("Hello world from processor %s, rank %d out of %d processors\n",
         processor_name, world_rank, world_size);

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

Initialize enironment

How many processors are there?

What is my rank?

What is my name?

Print name, rank, total processor count

Finalize after we are done

https://raw.githubusercontent.com/mpitutorial/mpitutorial/gh-pages/tutorials/mpi-hello-world/code/mpi_hello_world.c

PART 1 ENDED HERE

Announcements

- HW1 due today by 11:59pm
- HW2 out today right after class, deadline extended to Feb 5 (from Feb 3)
- Monday office hours moved to 1-2p due to room sharing schedule (updated on Canvas calendar)

Parallel Execution Model

The same MPI program is executed concurrently **on each process**

Process 0

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size); // Get the number
of processes

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank); // Get the rank of
the process

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len); // Get the
name of the processor

  printf("Hello world from processor %s, rank %d out of %d
processors\n",
         processor_name, world_rank, world_size); // Print hello
world message

  // Finalize the MPI environment. No more MPI calls can be made
after this
  MPI_Finalize();
}
```

Process 1

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size); // Get the number
of processes

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank); // Get the rank of
the process

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len); // Get the
name of the processor

  printf("Hello world from processor %s, rank %d out of %d
processors\n",
         processor_name, world_rank, world_size); // Print hello
world message

  // Finalize the MPI environment. No more MPI calls can be made
after this
  MPI_Finalize();
}
```

...

Process P-1

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  MPI_Init(NULL, NULL); // Initialize the MPI environment.

  int world_size;
  MPI_Comm_size(MPI_COMM_WORLD, &world_size); // Get the number
of processes

  int world_rank;
  MPI_Comm_rank(MPI_COMM_WORLD, &world_rank); // Get the rank of
the process

  char processor_name[MPI_MAX_PROCESSOR_NAME];
  int name_len;
  MPI_Get_processor_name(processor_name, &name_len); // Get the
name of the processor

  printf("Hello world from processor %s, rank %d out of %d
processors\n",
         processor_name, world_rank, world_size); // Print hello
world message

  // Finalize the MPI environment. No more MPI calls can be made
after this
  MPI_Finalize();
}
```

MPI Hello World Example Output

How to run it:

```
$ salloc -N2 --ntasks-per-node=4 -t1:00:00
salloc: Pending job allocation 1471
salloc: job 1471 queued and waiting for resources
$ srun mpi_hello_world
```

2 node * 4 processes per node = 8

Example output:

Order not guaranteed

```
Hello world from processor atl0, rank 0 out of 8 processors
Hello world from processor atl0, rank 2 out of 8 processors
Hello world from processor atl0, rank 3 out of 8 processors
Hello world from processor atl1, rank 4 out of 8 processors
Hello world from processor atl1, rank 7 out of 8 processors
Hello world from processor atl0, rank 1 out of 8 processors
Hello world from processor atl1, rank 5 out of 8 processors
Hello world from processor atl1, rank 6 out of 8 processors
```

https://gatech.service-now.com/home?id=kb_article_view&sysparm_article=KB0042096#mpi-jobs

Synchronization

- Many parallel algorithms require that **no process proceeds before all the processes** have reached the same state at certain points of a program.

- Explicit synchronization :

```
int MPI_Barrier (MPI_Comm comm)
```

- Implicit synchronization through use of e.g. pairs of `MPI_Send` and `MPI_Recv`.

- Ask yourself the following question: "*If Process 1 progresses 100 times faster than Process 2, will the final result still be correct?*"

Example: Ordered Output

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  // init as before

  for(int i = 0; i < world_size; i++) {
    MPI_Barrier(MPI_COMM_WORLD);
    if (i == world_rank) {
      printf("Hello world from processor %s, rank %d out of %d processors\n",
           processor_name, world_rank, world_size); // Print hello world message
      fflush(stdout);
    }
  }

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

Iterate over processor count

Example: Ordered Output

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  // init as before

  for(int i = 0; i < world_size; i++) {
    MPI_Barrier(MPI_COMM_WORLD);
    if (i == world_rank) {
      printf("Hello world from processor %s, rank %d out of %d processors\n",
           processor_name, world_rank, world_size); // Print hello world message
      fflush(stdout);
    }
  }

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

Iterate over processor count

Synchronization

Example: Ordered Output

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  // init as before

  for(int i = 0; i < world_size; i++) {
    MPI_Barrier(MPI_COMM_WORLD);
    if (i == world_rank) {
      printf("Hello world from processor %s, rank %d out of %d processors\n",
           processor_name, world_rank, world_size); // Print hello world message
      fflush(stdout);
    }
  }

  // Finalize the MPI environment. No more MPI calls can be made after this
  MPI_Finalize();
}
```

Iterate over processor count

Synchronization

Only print if we are on our rank

Example: Ordered Output

The processes synchronize between themselves P times:

Process 0

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  // init as before

    for(int i = 0; i < size; i++) {
         MPI_Barrier(MPI_COMM_WORLD);
         if (i == world_rank) {
           printf("Hello world from processor %s, rank %d out of
         %d processors\n",
             processor_name, world_rank, world_size); // Print
    hello world message
              fflush(stdout);
         }
    }

  // Finalize the MPI environment. No more MPI calls can be made
after this
  MPI_Finalize();
}
```

Process 1

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  // init as before

    for(int i = 0; i < size; i++) {
         MPI_Barrier(MPI_COMM_WORLD);
         if (i == world_rank) {
           printf("Hello world from processor %s, rank %d out of
         %d processors\n",
             processor_name, world_rank, world_size); // Print
    hello world message
              fflush(stdout);
         }
    }

  // Finalize the MPI environment. No more MPI calls can be made
after this
  MPI_Finalize();
}
```

...

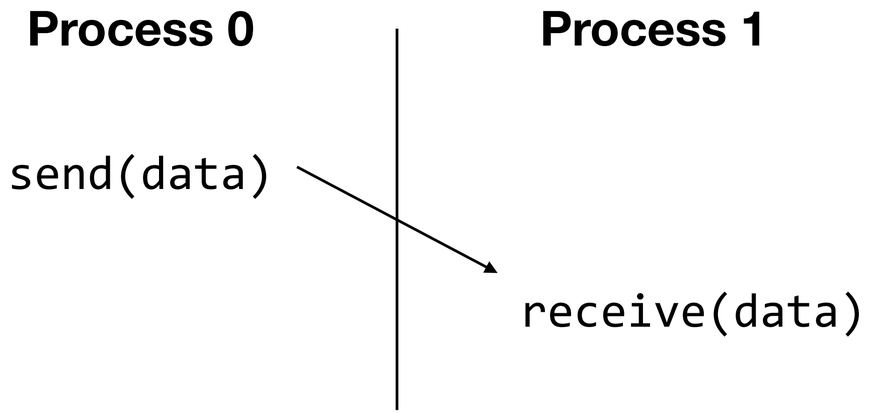
Process P-1

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
  // init as before

    for(int i = 0; i < size; i++) {
         MPI_Barrier(MPI_COMM_WORLD);
         if (i == world_rank) {
           printf("Hello world from processor %s, rank %d out of
         %d processors\n",
             processor_name, world_rank, world_size); // Print
    hello world message
              fflush(stdout);
         }
    }

  // Finalize the MPI environment. No more MPI calls can be made
after this
  MPI_Finalize();
}
```

Output:

```
Hello world from processor atl0, rank 0 out of 8 processors
Hello world from processor atl0, rank 1 out of 8 processors
Hello world from processor atl0, rank 2 out of 8 processors
Hello world from processor atl0, rank 3 out of 8 processors
Hello world from processor atl1, rank 4 out of 8 processors
Hello world from processor atl1, rank 5 out of 8 processors
Hello world from processor atl1, rank 6 out of 8 processors
Hello world from processor atl1, rank 7 out of 8 processors
```

MPI Basic Send/Receive



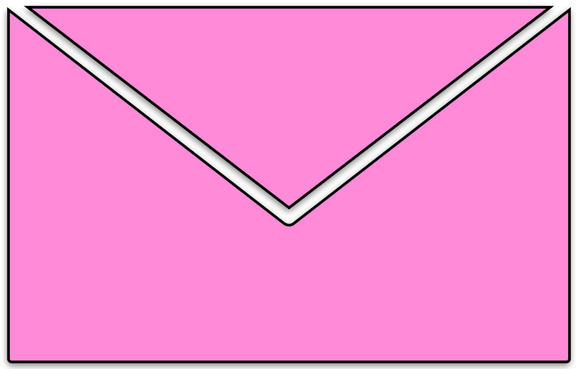
Things that need specifying:

- How will “data” be described?
- How will processes be identified?
- How will the receiver recognize/screen messages?
- What will it mean for these operations to complete?

MPI Message

An MPI message is an array of data elements "inside an envelope"

- Data: start address of the message buffer, counter of elements in the buffer, data type
- Envelope: source/destination process, message tag, communicator



MPI Datatypes

- The data in a message to send or receive is described by a triple (address, count, datatype), where

- An MPI datatype is recursively defined as:
  - **predefined**, corresponding to a data type from the language (e.g., `MPI_INT, MPI_DOUBLE`)
  - a contiguous array of MPI datatypes
  - a strided block of datatypes
  - an indexed array of blocks of datatypes
  - an arbitrary structure of datatypes

- There are MPI functions to construct custom datatypes, in particular ones for subarrays
- May hurt performance if datatypes are complex

The Simplest MPI Send Command

```
int MPI_Send(void *buf, int count,
             MPI_Datatype datatype,
             int dest, int tag,
             MPI_Comm comm)
```

Returns only when communication is finished

This **blocking send function** returns when the data has been delivered to the system and the buffer can be reused. The message may not have been received by the destination process.

The Simplest MPI Receive Command

```
int MPI_Recv(void *buf, int count
             MPI_Datatype datatype,
             int source, int tag,
             MPI_Comm comm,
             MPI_Status *status);
```

- This **blocking receive function** waits until a matching message is received from the system so that the buffer contains the incoming message.
- Match of data type, source process (or `MPI_ANY_SOURCE`), message tag (or `MPI_ANY_TAG`).
- Receiving fewer `datatype` elements than count is ok, but receiving more is an error.

Assuming 2 processes

A Simple MPI Send/Receive Program

```
#include “mpi.h”
#include <stdio.h>

int main( int argc, char *argv[]) {
  int rank, buf;
  MPI_Status status;
  MPI_Init(&argv, &argc);
  MPI_Comm_rank( MPI_COMM_WORLD, &rank );

  /* Process 0 sends and Process 1 receives
  if (rank == 0) {
    buf = 123456;
    MPI_Send( &buf, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
  } else if (rank == 1) {
    MPI_Recv( &buf, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, &status );
    printf( “Received %d\n ”, buf );
  }
  MPI_Finalize();
return 0;
}
```

```
int MPI_Send(void *buf, int count,
             MPI_Datatype datatype,
             int dest, int tag,
             MPI_Comm comm)
```

```
int MPI_Recv(void *buf, int count
             MPI_Datatype datatype,
             int source, int tag,
             MPI_Comm comm,
             MPI_Status *status);
```

MPI Tags

```
int MPI_Send(void *buf, int count,
             MPI_Datatype datatype,
             int dest, int tag,
             MPI_Comm comm)
```

```
int MPI_Recv(void *buf, int count
             MPI_Datatype datatype,
             int source, int tag,
             MPI_Comm comm,
             MPI_Status *status);
```

Messages are sent with an accompanying user-defined integer tag, to assist the receiving process in identifying the message.

Messages can be screened at the receiving end by specifying a specific tag, or not screened by specifying `MPI_ANY_TAG` as the tag in a receive.

https://mpitutorial.com/tutorials/dynamic-receiving-with-mpi-probe-and-mpi-status/

MPI Status

```
int MPI_Recv(void *buf, int count
             MPI_Datatype datatype,
             int source, int tag,
             MPI_Comm comm,
             MPI_Status *status);
```

In C, the `MPI_Status` is a structure that contains at least 3 attributes: `MPI_SOURCE`, `MPI_TAG` and `MPI_ERROR`.

- `MPI_SOURCE` contains the rank of the process that sent the message.
- `MPI_TAG` contains the tag of the message received.
- `MPI_ERROR` contains the error code of the receive operation.

Querying `MPI_Status`

```
MPI_Get_count(
    MPI_Status* status,
    MPI_Datatype datatype,
    int* count)
```

In `MPI_Get_count`, the user passes the `MPI_Status` structure, the `datatype` of the message, and `count` is returned. The `count` variable is the total number of `datatype` elements that were received.

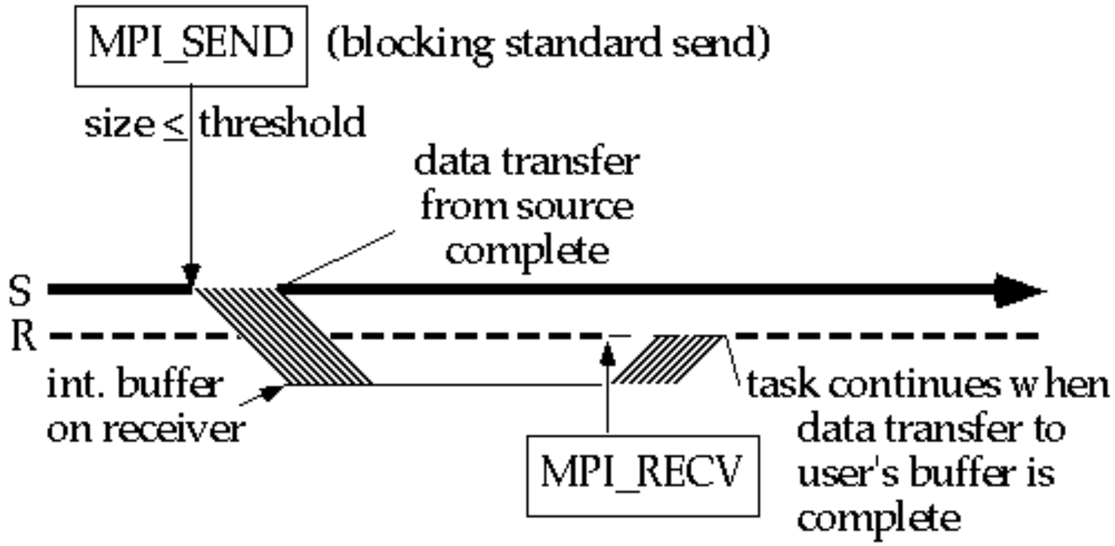
`MPI_Recv` can take `MPI_ANY_SOURCE` for the rank of the sender and `MPI_ANY_TAG` for the tag, so `MPI_Status` would be the only way to find the sender and tag in that case.

Blocking and Non-Blocking Communication

So far we have been using blocking communication:

- MPI_Recv does not complete until the buffer is full (available for use).
- MPI_Send does not complete until the buffer is empty (available for use).

Completion depends on size of message and amount of system buffering.



https://www.codingame.com/playgrounds/47058/have-fun-with-mpi-in-c/blocking-communication

Deadlocks

Send a large message from process 0 to process 1

- If there is insufficient storage at the destination, the send must wait for the user to provide the memory space (through a receive)

Process 0	Process 1
`Send(1)`	`Send(0)`
`Recv(1)`	`Recv(0)`

May deadlock if both processes are waiting for the other to finish sending

This is called “unsafe” because it **depends on the availability of system buffers** in which to store the data sent until it can be received.

Some Solutions to Deadlock in MPI

Order the operations more carefully:

Process 0	Process 1
`Send(1)`	`Recv(0)`
`Recv(1)`	`Send(0)`

Supply receive buffer at same time as send:

Process 0	Process 1
`Sendrecv(1)`	`Sendrecv(0)`

Can think of it as both send and receive executed concurrently

MPI_Sendrecv

```
int MPI_Sendrecv(const void* buffer_send,
                 int count_send,
                 MPI_Datatype datatype_send,
                 int recipient,
                 int tag_send,
                 void* buffer_recv,
                 int count_recv,
                 MPI_Datatype datatype_recv,
                 int sender,
                 int tag_recv,
                 MPI_Comm communicator,
                 MPI_Status* status);
```

- `MPI_Sendrecv` is a combination of an `MPI_send` and `MPI_Recv`.
- It can be seen as both subroutines executing concurrently.
- The buffers for send and receive must be different.

https://rookiehpc.org/mpi/docs/mpi_sendrecv/index.html

MPI_Sendrecv Example

```
// NOT SHOWN - INIT AND MAKE SURE YOU HAVE TWO PROCESSES

    // Prepare parameters
    int my_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
    int buffer_send = (my_rank == 0) ? 12345 : 67890;
    int buffer_recv;
    int tag_send = 0;
    int tag_recv = tag_send;
    int peer = (my_rank == 0) ? 1 : 0;

    // Issue the send + receive at the same time
    printf("MPI process %d sends value %d to MPI process %d.\n", my_rank, buffer_send, peer);
    MPI_Sendrecv(&buffer_send, 1, MPI_INT, peer, tag_send,
                 &buffer_recv, 1, MPI_INT, peer, tag_recv, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("MPI process %d received value %d from MPI process %d.\n", my_rank, buffer_recv,
peer);

// NOT SHOWN - FINALIZE
```

https://rookiehpc.org/mpi/docs/mpi_sendrecv/index.html

MPI_Sendrecv Example

```
// NOT SHOWN - INIT AND MAKE SURE YOU HAVE TWO PROCESSES

    // Prepare parameters
    int my_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
    int buffer_send = (my_rank == 0) ? 12345 : 67890;
    int buffer_recv;
    int tag_send = 0;
    int tag_recv = tag_send;
    int peer = (my_rank == 0) ? 1 : 0;

    // Issue the send + receive at the same time
    printf("MPI process %d sends value %d to MPI process %d.\n", my_rank, buffer_send, peer);
    MPI_Sendrecv(&buffer_send, 1, MPI_INT, peer, tag_send,
                 &buffer_recv, 1, MPI_INT, peer, tag_recv, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("MPI process %d received value %d from MPI process %d.\n", my_rank, buffer_recv,
peer);

// NOT SHOWN - FINALIZE
```

P0 sends 12345, receives 67890
P1 sends 67890, receives 12345

https://rookiehpc.org/mpi/docs/mpi_sendrecv/index.html

Another Solution: Non-Blocking Send/Receive

Non-blocking operations **return (immediately)** “request handles” that can be tested and waited on:

- `MPI_Request request;`
- `MPI_Status status;`
- `MPI_Isend(…);`
- `MPI_Irecv(…);`
- `MPI_Wait(&request, &status);` (each request must be Waited on)

One can also test without waiting:

```
MPI_Test(&request, &flag, &status);
```

Accessing the data buffer without waiting is undefined

MPI_Isend

```
int MPI_Isend(const void* buffer,
              int count,
              MPI_Datatype datatype,
              int recipient,
              int tag,
              MPI_Comm communicator,
              MPI_Request* request);
```

Handle on non-blocking operation (used later to check for completion)

- `MPI_Isend` is the standard **non-blocking send** (I stands for immediate return).
- The user must not attempt to reuse the buffer after `MPI_Isend` returns without explicitly **checking for completion**.

https://rookiehpc.org/mpi/docs/mpi_isend/index.html

`MPI_Wait` and `MPI_Test`

```
int MPI_Wait(MPI_Request* request,
             MPI_Status* status);
```

`MPI_Wait` **waits** for a non-blocking operation to complete and will block until the underlying operation is done.

```
int MPI_Test(MPI_Request* request,
             int* flag,
             MPI_Status* status);
```

`MPI_Test` checks if a non-blocking operation is complete at a given time.
It will not wait for the underlying non-blocking operation to complete.

https://rookiehpc.org/mpi/docs/mpi_wait/index.html
https://rookiehpc.org/mpi/docs/mpi_test/index.html

Multiple Completions

It is sometimes desirable to **wait on multiple requests**:

```
MPI_Waitall(count, array_of_requests, array_of_statuses)
MPI_Waitany(count, array_of_requests, &index, &status)
MPI_Waitsome(count, array_of_requests, array_of_indices,
array_of_statuses)
```

There are corresponding versions of test for each of these.

Two processes

MPI_Isend Example - Sender

```
   switch(my_rank)
    {
        case SENDER:
        {
            int buffer_sent = 12345;
            MPI_Request request;
            printf("MPI process %d sends value %d.\n", my_rank, buffer_sent);
            MPI_Isend(&buffer_sent, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &request);

            // Do other things while the MPI_Isend completes
            // <...>

            // Let's wait for the MPI_Isend to complete before progressing
further.
            MPI_Wait(&request, MPI_STATUS_IGNORE);
            break;
        }
        // RECEIVER CASE ON NEXT SLIDE
    }
```

https://rookiehpc.org/mpi/docs/mpi_isend/index.html

Two processes

MPI_Isend Example - Sender

```
   switch(my_rank)
    {
        case SENDER:
        {
            int buffer_sent = 12345;
            MPI_Request request;
            printf("MPI process %d sends value %d.\n", my_rank, buffer_sent);
            MPI_Isend(&buffer_sent, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &request);

            // Do other things while the MPI_Isend completes
            // <...>

            // Let's wait for the MPI_Isend to complete before progressing
further.
            MPI_Wait(&request, MPI_STATUS_IGNORE);
            break;
        }
        // RECEIVER CASE ON NEXT SLIDE
    }
```

MPI process 0 sends value 12345

https://rookiehpc.org/mpi/docs/mpi_isend/index.html

Two processes

MPI_Isend Example - Sender

```
   switch(my_rank)
    {
        case SENDER:
        {
            int buffer_sent = 12345;
            MPI_Request request;
            printf("MPI process %d sends value %d.\n", my_rank, buffer_sent);
            MPI_Isend(&buffer_sent, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, &request);

            // Do other things while the MPI_Isend completes
            // <...>

            // Let's wait for the MPI_Isend to complete before progressing
further.
            MPI_Wait(&request, MPI_STATUS_IGNORE);
            break;
        }
        // RECEIVER CASE ON NEXT SLIDE
    }
```

MPI process 0 sends value 12345

Only continue after the send is done

https://rookiehpc.org/mpi/docs/mpi_isend/index.html

Two processes

MPI_Isend Example - Receiver

Does not need to be a special recv

```
switch(my_rank)
 {
      // CASE SENDER FROM BEFORE NOT SHOWN
     case RECEIVER:
     {
         int received;
         MPI_Recv(&received, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
         printf("MPI process %d received value: %d.\n", my_rank, received);
         break;
     }
 }
```

MPI process 1 received value 12345

https://rookiehpc.org/mpi/docs/mpi_isend/index.html

MPI_Irecv

```
int MPI_Irecv(void* buffer,
              int count,
              MPI_Datatype datatype,
              int sender,
              int tag,
              MPI_Comm communicator,
              MPI_Request* request);
```

Handle on non-blocking operation (used later to check for completion)

- MPI_Isend is the standard **non-blocking receive** (I stands for immediate return).

- To know whether the message has been received, you must use `MPI_Wait` or `MPI_test` on the `MPI_Request`.

https://rookiehpc.org/mpi/docs/mpi_irecv/index.html

MPI_Irecv Example

```
    switch(my_rank)
    {
        // NOT SHOWN - The primary MPI process sends the value 12345 synchronously.
        case RECEIVER:
        {
            // The secondary MPI process receives the message.
            int received;
            MPI_Request request;
            printf("[Process %d] I issue the MPI_Irecv to receive the message as a
background task.\n", my_rank);
            MPI_Irecv(&received, 1, MPI_INT, SENDER, 0, MPI_COMM_WORLD, &request);

            // Do other things while the MPI_Irecv completes.
            printf("[Process %d] The MPI_Irecv is issued, I now moved on to print this
message.\n", my_rank);

            // Wait for the MPI_Recv to complete.
            MPI_Wait(&request, MPI_STATUS_IGNORE);
            printf("[Process %d] The MPI_Irecv completed, therefore so does the underlying
MPI_Recv. I received the value %d.\n", my_rank, received);
            break;
        }
    }
```

https://rookiehpc.org/mpi/docs/mpi_irecv/index.html

MPI_Irecv Example

```
    switch(my_rank)
    {
        // NOT SHOWN - The primary MPI process sends the value 12345 synchronously.
        case RECEIVER:
        {
            // The secondary MPI process receives the message.
            int received;
            MPI_Request request;
            printf("[Process %d] I issue the MPI_Irecv to receive the message as a
background task.\n", my_rank);
            MPI_Irecv(&received, 1, MPI_INT, SENDER, 0, MPI_COMM_WORLD, &request);

            // Do other things while the MPI_Irecv completes.
            printf("[Process %d] The MPI_Irecv is issued, I now moved on to print this
message.\n", my_rank);

            // Wait for the MPI_Recv to complete.
            MPI_Wait(&request, MPI_STATUS_IGNORE);
            printf("[Process %d] The MPI_Irecv completed, therefore so does the underlying
MPI_Recv. I received the value %d.\n", my_rank, received);
            break;
        }
    }
```

Can do local work while waiting

https://rookiehpc.org/mpi/docs/mpi_irecv/index.html

MPI_Irecv Example

```
    switch(my_rank)
    {
        // NOT SHOWN - The primary MPI process sends the value 12345 synchronously.
        case RECEIVER:
        {
            // The secondary MPI process receives the message.
            int received;
            MPI_Request request;
            printf("[Process %d] I issue the MPI_Irecv to receive the message as a
background task.\n", my_rank);
            MPI_Irecv(&received, 1, MPI_INT, SENDER, 0, MPI_COMM_WORLD, &request);

            // Do other things while the MPI_Irecv completes.
            printf("[Process %d] The MPI_Irecv is issued, I now moved on to print this
message.\n", my_rank);

            // Wait for the MPI_Recv to complete.
            MPI_Wait(&request, MPI_STATUS_IGNORE);
            printf("[Process %d] The MPI_Irecv completed, therefore so does the underlying
MPI_Recv. I received the value %d.\n", my_rank, received);
            break;
        }
    }
```

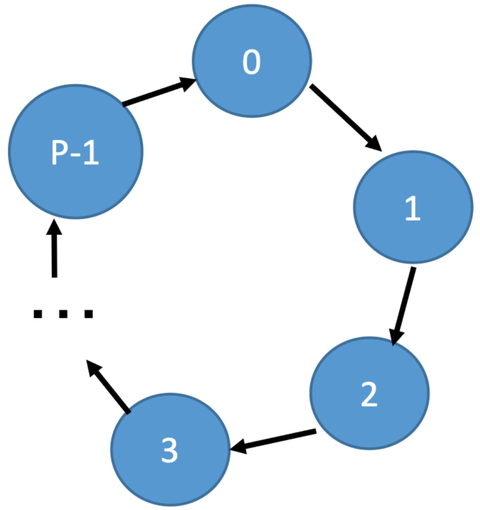
Can do local work while waiting

value 12345

https://rookiehpc.org/mpi/docs/mpi_irecv/index.html

Cyclic Dependencies

- Let's say we want to send messages according to right shift permutation:
  - Processor `i` sends a message to `(i+1) mod p`



Example:

```
MPI_Send(&x, 1, MPI_INT,
         (rank+1) % size, 13, MPI_COMM_WORLD);
MPI_Recv(&y, 1, MPI_INT,
         MPI_ANY_SOURCE, 13, MPI_COMM_WORLD, &stat);
```

What's wrong?

May block till message is received, MPI_Recv never called

Using Non-blocking receive:

```
MPI_Request req;
MPI_Irecv(&y, 1, MPI_INT,
         MPI_ANY_SOURCE, 13, MPI_COMM_WORLD, &req);
MPI_Send(&x, 1, MPI_INT,
         (rank+1) % size, 13, MPI_COMM_WORLD);
MPI_Wait(&req, &stat);
```

Correct!

From Srinivas Aluru

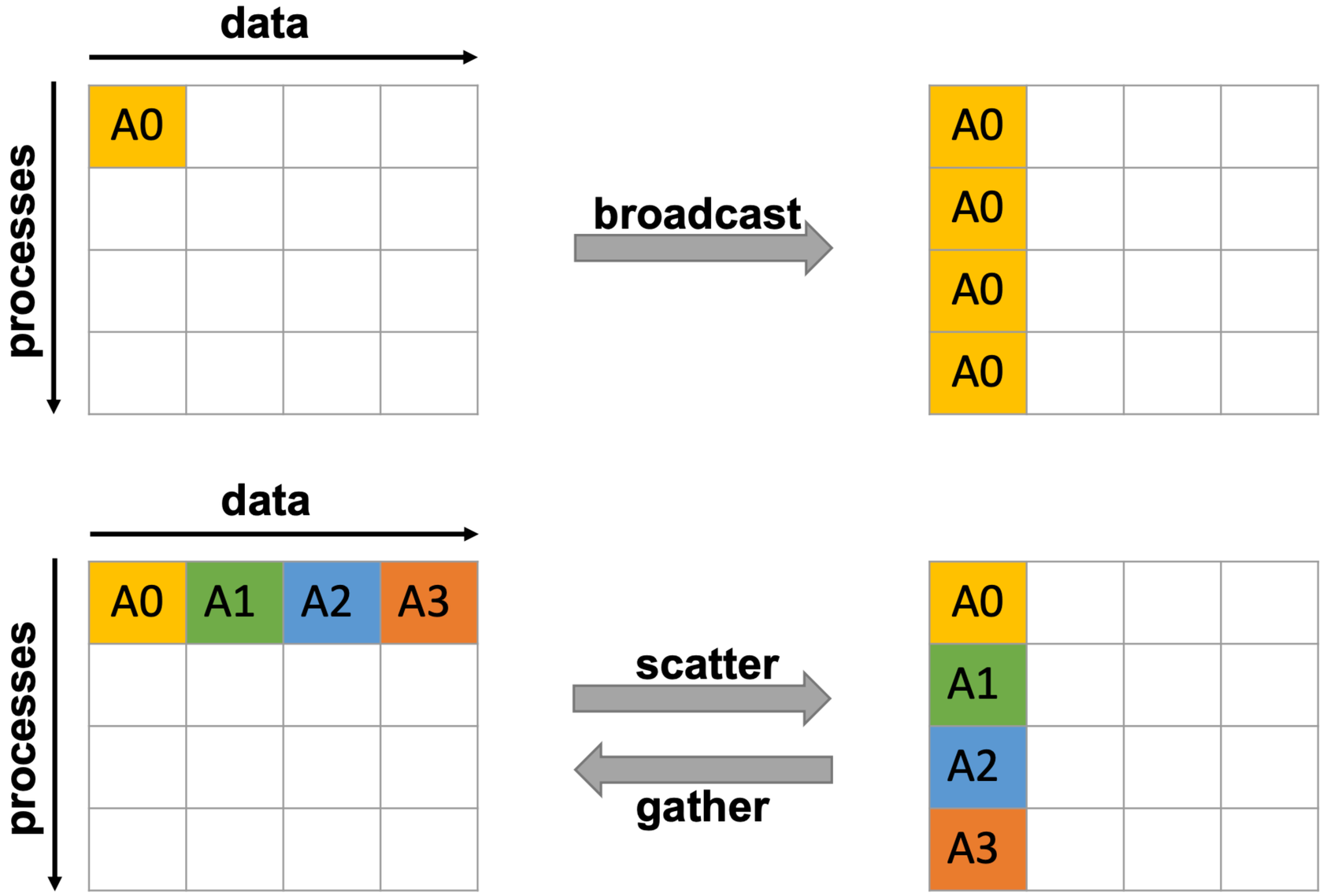
Introduction to Collectives
(there will be a whole lecture on this later)

Collective Communication

Summary:

- All processes within a communicator must participate
- All collective operations are **blocking**
- Except of `MPI_Barrier`, no synchronization can be assumed
- All collective operations are guaranteed to not interfere with point-to-point messages
- Collective operations can be implemented using only point-to-point calls

Broadcast, Scatter, Gather



From Srinivas Aluru

Reductions

- Let's assume we want to compute a sum over n elements:

```
int sum = 0;
for (int i = 0; i < n; ++i) sum = sum + A[i];
```

- Can we do this in parallel? -> Parallel Reductions

```
int MPI_Reduce(void* sbuf, void* rbuf, int count, MPI_Datatype type,
               MPI_Op op, int root, MPI_Comm comm);
```

Example:

```
// assume each processor has one element:
int a = A[rank];
int sum = 0;
MPI_Reduce(&a, &sum, 1, MPI_INT,
           MPI_SUM , 0, MPI_COMM_WORLD);
std::cout << sum << std::endl;
```

Summary

- MPI defines a standard for programming distributed-memory machines.
- The core functionality for message passing is send and receive (and variants).

MPI References

- MPI Standard: https://www.mpi-forum.org/
- Other information: https://www.mcs.anl.gov/research/projects/mpi/index.htm
- LLNL tutorial: https://hpc-tutorials.llnl.gov/mpi/
- https://mpitutorial.com/

Backup Slides & More Examples

Example of Querying `MPI_Status`

```
const int MAX_NUMBERS = 100;
int numbers[MAX_NUMBERS];
int number_amount;
if (world_rank == 0) {
    // Pick a random amount of integers to send to process one
    srand(time(NULL));
    number_amount = (rand() / (float)RAND_MAX) * MAX_NUMBERS;

    // Send the amount of integers to process one
    MPI_Send(numbers, number_amount, MPI_INT, 1, 0, MPI_COMM_WORLD);
    printf("0 sent %d numbers to 1\n", number_amount);
}
…
```

https://mpitutorial.com/tutorials/dynamic-receiving-with-mpi-probe-and-mpi-status/

Example of Querying MPI_Status

```
…
} else if (world_rank == 1) {
    MPI_Status status;
    // Receive at most MAX_NUMBERS from process zero
    MPI_Recv(numbers, MAX_NUMBERS, MPI_INT, 0, 0, MPI_COMM_WORLD,
             &status);

    // After receiving the message, check the status to determine
    // how many numbers were actually received
    MPI_Get_count(&status, MPI_INT, &number_amount);

    // Print off the amount of numbers, and also print additional
    // information in the status object
    printf("1 received %d numbers from 0. Message source = %d, "
           "tag = %d\n",
           number_amount, status.MPI_SOURCE, status.MPI_TAG);
}
```

Output Example:

```
0 sent 93 numbers to 1
1 dynamically received 93 numbers from 0
```

More MPI Send/Receive Examples

```
int ping_pong_count = 0;
int partner_rank = (world_rank + 1) % 2;
while (ping_pong_count < PING_PONG_LIMIT) {
    if (world_rank == ping_pong_count % 2) {
        // Increment the ping pong count before you send it
        ping_pong_count++;
        MPI_Send(&ping_pong_count, 1, MPI_INT, partner_rank, 0,
                 MPI_COMM_WORLD);
        printf("%d sent and incremented ping_pong_count "
               "%d to %d\n", world_rank, ping_pong_count,
               partner_rank);
    } else {
        MPI_Recv(&ping_pong_count, 1, MPI_INT, partner_rank, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("%d received ping_pong_count %d from %d\n",
               world_rank, ping_pong_count, partner_rank);
    }
}
```

MPI Ping Pong Output

```
0 sent and incremented ping_pong_count 1 to 1
0 received ping_pong_count 2 from 1
0 sent and incremented ping_pong_count 3 to 1
0 received ping_pong_count 4 from 1
0 sent and incremented ping_pong_count 5 to 1
0 received ping_pong_count 6 from 1
0 sent and incremented ping_pong_count 7 to 1
0 received ping_pong_count 8 from 1
0 sent and incremented ping_pong_count 9 to 1
0 received ping_pong_count 10 from 1
1 received ping_pong_count 1 from 0
1 sent and incremented ping_pong_count 2 to 0
1 received ping_pong_count 3 from 0
1 sent and incremented ping_pong_count 4 to 0
1 received ping_pong_count 5 from 0
1 sent and incremented ping_pong_count 6 to 0
1 received ping_pong_count 7 from 0
1 sent and incremented ping_pong_count 8 to 0
1 received ping_pong_count 9 from 0
1 sent and incremented ping_pong_count 10 to 0
```

MPI Ring Send/Receive Example

```
int token;
if (world_rank != 0) {
    MPI_Recv(&token, 1, MPI_INT, world_rank - 1, 0,
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Process %d received token %d from process %d\n",
           world_rank, token, world_rank - 1);
} else {
    // Set the token's value if you are process 0
    token = -1;
}
MPI_Send(&token, 1, MPI_INT, (world_rank + 1) % world_size,
         0, MPI_COMM_WORLD);

// Now process 0 can receive from the last process.
if (world_rank == 0) {
    MPI_Recv(&token, 1, MPI_INT, world_size - 1, 0,
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
    printf("Process %d received token %d from process %d\n",
           world_rank, token, world_size - 1);
}
```

https://mpitutorial.com/tutorials/mpi-send-and-receive/

MPI Ring Send/Receive Output

```
Process 1 received token -1 from process 0
Process 2 received token -1 from process 1
Process 3 received token -1 from process 2
Process 4 received token -1 from process 3
Process 0 received token -1 from process 4
```